

TJSC 2026 - Analista de Sistemas

Segurança da Informação e Governança de TI

Aula 03

IAM, autenticação, autorização, MFA, SSO, OAuth2, OpenID Connect e SAML

Material autoral para revisão objetiva, com exemplos de APIs, Java/Quarkus, Angular, sistemas corporativos e ambiente bancário. Questões autorais simulam o estilo FGV, sem reprodução de questões reais. O foco é converter vivência prática em acerto de prova.

Objetivo de prova: dominar a diferença entre autenticação, autorização, gestão de identidade, federação, tokens, claims e protocolos de SSO, evitando as confusões que a FGV costuma explorar.

1. Como esta aula deve ser estudada

Esta aula fica no centro do bloco de Segurança da Informação do edital. Ela conversa com criptografia, APIs, desenvolvimento seguro, LGPD e governança, mas o núcleo é: como uma organização reconhece usuários, concede acesso, propaga identidade entre sistemas e controla sessões e tokens.

Para ganhar ROI na FGV, a prioridade não é decorar todas as variações de implementação. A prioridade é diferenciar termos que parecem iguais: autenticação, autorização, identidade, credencial, autenticador, sessão, token, claim, assertion, IdP, SP, RP, access token, ID token, refresh token, OAuth2, OpenID Connect e SAML.

Regra de ouro: autenticação responde 'quem é você?'; autorização responde 'o que você pode fazer?'; federação responde 'como um sistema confia na autenticação feita por outro sistema?'; IAM organiza tudo isso como processo corporativo.

1.1. Ponto de partida: mapa mental textual

- IAM - Identity and Access Management: disciplina corporativa para administrar identidades, contas, papéis, permissões, autenticação, autorização, revisão de acesso e trilhas de auditoria.
- Autenticação: prova de identidade, geralmente por senha, certificado, token físico, biometria, aplicativo autenticador ou chave criptográfica.
- Autorização: decisão de acesso após a autenticação, normalmente por papéis, atributos, políticas, escopos ou permissões.
- MFA - Multi-Factor Authentication: autenticação com fatores de naturezas diferentes, como algo que você sabe + algo que você possui.
- SSO - Single Sign-On: experiência de login único para acessar vários sistemas confiando em um provedor de identidade.
- OAuth 2.0: framework de autorização delegada; permite acesso limitado a recursos sem entregar a senha ao cliente.
- OpenID Connect - OIDC: camada de identidade sobre OAuth 2.0; acrescenta autenticação padronizada e ID Token.
- SAML - Security Assertion Markup Language: padrão XML de federação e SSO, muito usado em ambientes corporativos e aplicações legadas/enterprise.

1.2. Siglas e termos da aula

Sigla/termo	Expansão	Associação de prova
IAM	Identity and Access Management	Guarda-chuva: identidade + acesso + ciclo de vida + governança de permissões.
IdP	Identity Provider	Provedor de identidade. Autentica o usuário e emite tokens/assertions.
SP	Service Provider	Termo comum em SAML. Serviço que consome a assertion do IdP.
RP	Relying Party	Termo comum em OIDC. Aplicação que confia no provedor OpenID.
SSO	Single Sign-On	Login único; não é 'mesma senha em tudo'.
MFA	Multi-Factor Authentication	Dois ou mais fatores independentes de autenticação.
OAuth	Open Authorization	Autorização delegada. Não é protocolo de autenticação puro.
OIDC	OpenID Connect	Autenticação federada baseada em OAuth 2.0 + ID Token.
SAML	Security Assertion Markup Language	Assertions XML assinadas entre IdP e SP.

Sigla/termo	Expansão	Associação de prova
JWT	JSON Web Token	Formato compacto para claims; assinado não significa criptografado.
JWS	JSON Web Signature	JWT com assinatura/integridade.
JWE	JSON Web Encryption	JWT cifrado, com confidencialidade.
TOTP	Time-Based One-Time Password	Código temporário de app autenticador, geralmente de 30 em 30 segundos.
HOTP	HMAC-Based One-Time Password	Código de uso único baseado em contador.
PKCE	Proof Key for Code Exchange	Protege o Authorization Code Flow, principalmente clientes públicos como SPA/mobile.
RBAC	Role-Based Access Control	Permissões por papéis: gerente, analista, administrador.
ABAC	Attribute-Based Access Control	Permissões por atributos: lotação, horário, local, risco, cargo, classificação.
DAC	Discretionary Access Control	Dono do recurso decide permissões.
MAC	Mandatory Access Control	Política central obrigatória; típico de ambientes mais rígidos.

2. Vocabulário básico sem assumir que você lembra

Em Segurança da Informação, a FGV explora termos pequenos com impacto grande. Abaixo estão as palavras que mais confundem em autenticação, autorização e federação.

Termo	O que é	Como memorizar
Identidade	Representação de uma pessoa, sistema ou serviço no ambiente digital.	'Quem existe no sistema'. Ex.: matrícula funcional, CPF, conta técnica.
Conta	Registro técnico associado a uma identidade em um sistema.	A pessoa é Jonas; a conta é j123456 no AD, Keycloak ou aplicação.
Credencial	Informação ou objeto usado para provar identidade.	Senha, certificado, chave privada, token físico, segredo de app.
Autenticador	Meio que o usuário controla para se autenticar.	App OTP, chave FIDO2, certificado, senha, smartcard.
Sessão	Estado mantido após autenticação para evitar novo login a cada requisição.	Login gera sessão; sessão expira, pode ser revogada e tem cookie/token.
Token	Objeto digital que representa autorização, identidade ou sessão.	'Crachá digital'. Pode ter escopo, validade e emissor.
Claim	Afirmação dentro de um token/assertion.	'Este usuário tem sub=123, aud=api, role=admin'.
Assertion	Afirmação de segurança emitida por IdP, muito associada ao SAML.	O IdP 'afirma' ao SP que o usuário autenticou.
Nonce	Número/valor usado uma vez para evitar repetição/replay.	'once' em inglês ajuda: n-once. Serve para frescor.
State	Valor usado em OAuth/OIDC para correlacionar requisição e resposta e mitigar CSRF.	Estado da conversa: o que saiu deve voltar igual.
Salt	Valor aleatório adicionado a senha antes do hash para impedir tabelas pré-computadas e hashes iguais para senhas iguais.	Tempero individual da senha antes do hash. Não é segredo; precisa ser único/aleatório.
Scope	Escopo de autorização em OAuth2, limita o que o cliente pode fazer.	'Pode ler conta', 'pode criar pagamento', 'pode ver e-mail'.
Audience - aud	Destinatário esperado de um token.	Evita token emitido para API A ser aceito pela API B.

Associação rápida: salt protege armazenamento de senha; nonce protege contra repetição; state protege o fluxo de autorização contra troca/CSRF; scope limita o poder do token; aud limita o destinatário que deve aceitar o token.

3. Autenticação x autorização x auditoria

3.1. Autenticação: provar identidade

Autenticação é o processo de verificar se o sujeito que tenta acessar um serviço controla um ou mais autenticadores associados a uma identidade. Em termos práticos: o sistema quer saber se aquela pessoa ou aplicação é realmente quem diz ser.

No seu mundo de APIs e sistemas corporativos, isso aparece quando um usuário entra via provedor corporativo, quando um serviço usa certificado ou client credential para chamar outro serviço, ou quando uma aplicação exige MFA para operações sensíveis.

3.2. Autorização: decidir acesso

Autorização vem depois, ou em conjunto, mas conceitualmente é diferente. Um usuário autenticado pode não estar autorizado. Exemplo: o usuário logou corretamente no sistema de pagamentos, mas não tem permissão para aprovar pagamento acima de determinado valor.

Em APIs REST, a autorização costuma aparecer em verificações de escopo, papel, atributo, política de negócio e ownership do recurso. Um erro clássico de prova e de sistema real é autenticar bem, mas autorizar mal. Isso gera vulnerabilidades como acesso indevido horizontal, em que um usuário acessa dados de outro usuário alterando um identificador na URL.

Conceito	Pergunta que responde	Exemplo em sistema corporativo	Erro comum de prova
Autenticação	Quem é você?	Login com senha + MFA no IdP corporativo.	Dizer que OAuth2 é autenticação pura.
Autorização	O que você pode fazer?	Escopo permite consultar, mas não aprovar pagamento.	Confundir usuário logado com usuário autorizado.
Accounting/Auditoria	O que você fez?	Logs de aprovação, IP, horário, trilha de alteração.	Esquecer que controle também envolve rastreabilidade.

3.3. AAA: Authentication, Authorization and Accounting

AAA significa Authentication, Authorization and Accounting. É uma sigla útil para provas porque organiza três etapas: autenticar, autorizar e registrar/prestar contas. Em redes, aparece com RADIUS e TACACS+; em sistemas corporativos, aparece como login, autorização por perfis e log/auditoria.

Pegadinha FGV: autenticação forte sem boa autorização não impede acesso indevido. MFA reduz risco de roubo de credenciais, mas não corrige uma API que não valida se o recurso pertence ao usuário.

4. IAM - Identity and Access Management

4.1. O que IAM cobre

IAM - Identity and Access Management **é a disciplina que administra o ciclo de vida das identidades e dos acessos**. Em ambiente público, bancário ou judiciário, isso envolve cadastro, concessão, alteração, revisão e revogação de acessos, com segregação de funções, trilhas de auditoria e conformidade.

IAM não é apenas a tela de login. **É governança de acesso**. Inclui quem pode criar usuários, como perfis são concedidos, como acessos são revisados, o que acontece quando alguém muda de lotação, como contas privilegiadas são monitoradas e como integrações entre sistemas propagam identidade.

4.2. Ciclo de vida de identidade

Etapa	O que acontece	Risco se mal implementada
Joiner	Entrada: criação da identidade e concessão inicial de acesso.	Acesso excessivo desde o início.
Mover	Mudança de área, cargo, projeto ou função.	Acúmulo de permissões antigas + novas.
Leaver	Desligamento, afastamento ou fim de contrato.	Conta ativa sem necessidade; risco de uso indevido.
Recertificação	Revisão periódica de acessos por gestores/donos de sistemas.	Permissões órfãs e incompatíveis com a função.
Revogação	Remoção de acesso ou encerramento de sessão/token.	Usuário mantém acesso depois de perder autorização.

4.3. Modelos de controle de acesso

Modelo	Ideia central	Exemplo	Pegadinha
RBAC - Role-Based Access Control	Permissões associadas a papéis/cargos.	Papel APROVADOR_PAGAMENTO ou ANALISTA_CONSULTA.	Não é sempre granular; pode gerar papéis demais.
ABAC - Attribute-Based Access Control	Decisão por atributos do usuário, recurso, ambiente e ação.	Permite aprovar só se lotação=X, valor<limite, horário comercial e risco baixo.	É mais dinâmico, mas exige boa governança de atributos.
DAC - Discretionary Access Control	Dono do recurso concede permissões.	Dono de pasta compartilha com outro usuário.	Flexível, mas pode perder controle central.
MAC - Mandatory Access Control	Política central obrigatória impõe rótulos/níveis.	Documento secreto só para usuários com clearance adequado.	Mais rígido; não depende da vontade do dono do arquivo.
PBAC - Policy-Based Access Control	Políticas expressam regras de acesso.	Motor de políticas decide com base em regras versionadas.	Pode ser visto como evolução/implementação baseada em atributos e políticas.

Para FGV: RBAC = papel; ABAC = atributos; DAC = dono decide; MAC = política central obrigatória. Em casos situacionais, procure no enunciado quem decide a permissão: dono, política central, papel ou atributos.

5. MFA e fatores de autenticação

5.1. Fatores: saber, possuir, ser

MFA - Multi-Factor Authentication combina fatores de autenticação independentes. A prova pode dizer 'duas etapas' e tentar induzir a erro: duas senhas não são dois fatores, pois ambas pertencem ao fator conhecimento. A robustez vem da diversidade de fatores.

Fator	Significado	Exemplos	Observação de prova
Conhecimento	Algo que você sabe.	Senha, PIN, resposta secreta.	Duas senhas continuam sendo um único tipo de fator.
Posse	Algo que você possui.	Token físico, smartphone com app OTP, smartcard, chave FIDO2.	SMS é posse fraca por riscos de SIM swap e interceptação.
Inerência	Algo que você é.	Biometria facial, impressão digital, íris.	Biometria é difícil de revogar; costuma ativar/disponibilizar um autenticador.

Fator	Significado	Exemplos	Observação de prova
Localização/comportamento	Contexto de acesso.	IP, geolocalização, dispositivo conhecido, horário.	Normalmente usado em autenticação adaptativa, não como fator clássico isolado.

5.2. OTP, TOTP, HOTP, push e FIDO

OTP - One-Time Password é uma senha de uso único. HOTP - HMAC-Based One-Time Password usa contador. TOTP - Time-Based One-Time Password usa tempo, por isso o código muda periodicamente. Aplicativos como autenticadores geram TOTP com base em um segredo compartilhado e no horário.

Push notification exige cuidado: aprovar cegamente uma notificação no celular pode viabilizar **MFA fatigue**, ataque em que o invasor tenta logins repetidos até o usuário aprovar uma notificação por cansaço ou distração. **FIDO2/WebAuthn** usa **criptografia assimétrica** e é mais resistente a phishing porque a chave se vincula ao domínio/origem legítima.

5.3. Níveis de garantia de autenticação - AAL

O NIST organiza a força da autenticação em AAL - Authentication Assurance Levels. O ponto de prova não é decorar cada requisito, e sim entender a progressão: AAL1 dá confiança básica, AAL2 exige dois fatores distintos, e AAL3 exige nível muito alto, com autenticador criptográfico baseado em hardware e resistência a personificação do verificador.

Nível	Ideia	Memorização
AAL1	Confiança básica; pode usar fator único.	Senha isolada pode existir aqui, embora MFA seja recomendável.
AAL2	Alta confiança; exige dois fatores distintos.	MFA real: conhecimento + posse, ou posse + biometria.
AAL3	Confiança muito alta; autenticação criptográfica com hardware e resistência a phishing/verificador falso.	Ambientes sensíveis, chaves físicas, smartcards, autenticação forte.

Não confunda MFA com SSO. MFA fortalece a prova de identidade. SSO melhora a experiência e centraliza confiança. Um ambiente pode ter SSO fraco, SSO forte com MFA, ou MFA sem SSO.

6. SSO e federação de identidade

6.1. SSO não é 'usar a mesma senha'

SSO - Single Sign-On é a capacidade de autenticar uma vez em um provedor confiável e acessar múltiplos serviços sem redigitar credenciais em cada um deles. O benefício operacional é reduzir fadiga de senha, centralizar políticas, facilitar revogação e melhorar auditoria.

Usar a mesma senha em vários sistemas não é SSO; é reutilização de credencial, uma prática insegura. No SSO verdadeiro, as aplicações delegam a autenticação a um provedor de identidade e confiam em tokens/assertions.

6.2. Federação: confiança entre domínios

Federação de identidade ocorre quando uma aplicação aceita afirmações de identidade emitidas por outro domínio de confiança. Em termos práticos: o sistema A não precisa guardar sua senha; ele confia no **IdP corporativo** que autenticou você e emitiu uma assertion ou token.

No ambiente do Banco, pense em um portal interno, uma aplicação Angular, um backend Quarkus e um provedor corporativo. O ideal é que cada aplicação não tenha sua própria senha local solta. Ela delega o login e valida token, escopo, issuer, audience, expiração e assinatura.

Elemento	Em SAML	Em OIDC/OAuth2	Função
Provedor de identidade	IdP	OpenID Provider / Authorization Server	Autentica e emite assertion/token.
Aplicação que confia	SP - Service Provider	RP - Relying Party / Client	Consome assertion/token e concede sessão local.
Objeto de prova	SAML Assertion	ID Token / Access Token	Transporta claims, identidade ou autorização.
Formato comum	XML	JSON/JWT	Como os dados são representados.

7. OAuth 2.0 - autorização delegada

7.1. Ideia central

OAuth 2.0 é um framework de autorização. Ele permite que um cliente obtenha acesso **limitado** a recursos protegidos sem receber a senha do usuário. A aplicação recebe um token com escopo e validade, e não a credencial primária do usuário.

Exemplo: um aplicativo quer consultar seu e-mail ou dados de perfil. Em vez de você entregar sua senha ao aplicativo, você autentica no provedor confiável, consente com um escopo e o cliente recebe um access token limitado. Em ambiente corporativo, a ideia vale para APIs internas: um frontend ou serviço acessa uma API mediante token emitido por servidor de autorização.

Frase de prova: **OAuth2 é autorização delegada**. Para autenticação padronizada de usuário em cima do OAuth2, pense em OpenID Connect.

7.2. Papéis do OAuth2

Papel	Explicação	Exemplo mental
Resource Owner	Dono do recurso protegido. Quando é pessoa, é o usuário final.	Servidor/usuário que autoriza acesso aos próprios dados.
Client	Aplicação que quer acessar recurso protegido.	SPA Angular, app mobile, backend Quarkus, serviço parceiro.
Authorization Server	Servidor que autentica, obtém autorização e emite tokens.	Keycloak, Azure AD/Entra ID, solução corporativa de IAM.
Resource Server	Servidor que hospeda o recurso protegido e valida access token.	API REST de pagamentos, API de cadastro, API de processos.

7.3. Tokens principais

Token	Para que serve	Quem consome	Pegadinha
Access Token	Autoriza acesso a recursos protegidos.	Resource Server/API.	Não é prova padronizada de identidade do usuário final.
Refresh Token	Permite obter novo access token sem nova interação do usuário.	Authorization Server.	Deve ser protegido; não é enviado à API de recurso.
ID Token	Prova autenticação e contém claims de identidade em OIDC.	Client/Relying Party.	Não deve ser usado como access token para API.
Authorization Code	Código temporário trocado por tokens no token endpoint.	Client + Authorization Server.	Deve ser curto, único e protegido com PKCE em clientes públicos.

7.4. Fluxos mais cobrados

O Authorization Code Flow é o fluxo mais importante para prova. Nele, o cliente redireciona o usuário ao Authorization Server, recebe um authorization code e troca esse código por tokens no token endpoint.

Em aplicações modernas, especialmente clientes públicos como SPA e mobile, usa-se PKCE - Proof Key for Code Exchange.

PKCE funciona como um 'comprovante de posse' do cliente que iniciou o fluxo. O cliente cria um code verifier, deriva um code challenge e envia o challenge na autorização. Depois, ao trocar o código por tokens, apresenta o verifier. Assim, se alguém interceptar o authorization code, não consegue trocá-lo sem o verifier.

Grant/fluxo	Uso típico	Situação atual de prova
Authorization Code + PKCE	Aplicações web, SPA e mobile modernas.	Preferível para usuário final; PKCE é essencial para cliente público.
Client Credentials	Comunicação máquina-máquina, sem usuário final.	Usado por serviço/backend chamando API com identidade própria.
Refresh Token	Renovar sessão/token sem novo login completo.	Requer proteção, rotação e revogação.
Device Authorization	Dispositivos com entrada limitada, TV/IoT.	Usuário autentica em outro dispositivo.
Implicit	SPA antiga recebia token direto no browser.	Hoje é desaconselhado/depreciado em boas práticas.
Resource Owner Password Credentials	Cliente coleta senha do usuário.	Evitar; contraria delegação e é ruim para SSO/MFA.

7.5. Scope, consentimento e menor privilégio

Scope é a unidade textual de autorização no OAuth2. Um token pode trazer escopos como read:processos, write:pagamentos ou profile. Em prova, scopes representam limitação de poder: o cliente não recebe acesso genérico ilimitado, mas um conjunto restrito de permissões.

No ambiente bancário e no TJSC, isso conversa diretamente com menor privilégio. Um sistema que consulta processos não deveria receber escopo de administração; um frontend não deveria receber refresh token de longa vida sem controle; uma API não deveria aceitar tokens sem validar assinatura, expiração, emissor e audiência.

8. OpenID Connect - OIDC

8.1. OIDC é identidade sobre OAuth2

OpenID Connect - OIDC é uma camada de identidade construída sobre OAuth 2.0. A diferença de prova é direta: OAuth2 responde melhor a 'o que esta aplicação pode acessar?'; OIDC responde também a 'quem é o usuário autenticado?'.

OIDC introduz o escopo openid e o ID Token. O ID Token é geralmente um JWT que contém claims sobre a autenticação do usuário final, como issuer, subject, audience, expiration, issued at e, dependendo do caso, nome, e-mail ou outros atributos.

8.2. ID Token x Access Token

Critério	ID Token	Access Token
Finalidade	Informar ao client/RP que o usuário foi autenticado e trazer claims de identidade.	Autorizar acesso a uma API/resource server.
Consumidor	Client/Relying Party.	Resource Server/API.
Pergunta respondida	Quem autenticou?	O que pode acessar?
Uso incorreto comum	Usar ID Token como credencial de API.	Usar access token como prova completa de identidade sem perfil OIDC.
Formato	Normalmente JWT assinado.	Pode ser opaco ou JWT, dependendo da implementação.

8.3. Claims que você deve reconhecer

Claim	Nome	Significado de prova
iss	Issuer	Quem emitiu o token. Deve bater com o provedor esperado.
sub	Subject	Identificador do usuário/sujeito no emissor.
aud	Audience	Para quem o token foi emitido. API/cliente deve validar.
exp	Expiration Time	Tempo de expiração. Token vencido deve ser recusado.
iat	Issued At	Quando foi emitido.
nbf	Not Before	Não aceitar antes de determinado instante.
nonce	Nonce	Evita replay/correlaciona autenticação no OIDC.
amr	Authentication Methods References	Métodos usados: senha, OTP, biometria etc.
acr	Authentication Context Class Reference	Classe/contexto de autenticação, útil para exigência de nível.

JWT assinado não é automaticamente secreto. Assinatura dá integridade e autenticidade da origem. Confidencialidade exige criptografia, como JWE, ou evitar colocar dados sensíveis no token.

9. SAML - Security Assertion Markup Language

9.1. O que SAML resolve

SAML - Security Assertion Markup Language é um padrão baseado em XML para trocar informações de autenticação e autorização entre domínios. É muito associado a SSO corporativo, especialmente em aplicações enterprise e integrações legadas.

No SAML, o usuário tenta acessar o Service Provider - SP; o SP redireciona ou solicita autenticação ao Identity Provider - IdP; o IdP autentica o usuário e emite uma SAML Assertion, normalmente assinada digitalmente. O SP valida a assertion e cria sessão local.

9.2. Componentes de uma assertion

Componente	Ideia	Pegadinha
Issuer	Quem emitiu a assertion.	SP deve confiar no IdP e validar assinatura.
Subject	Sobre quem a assertion fala.	Identidade do usuário/sujeito.
Conditions	Condições de validade.	Inclui janela temporal e audiência.
AuthnStatement	Declara como/quando ocorreu a autenticação.	Não é a senha do usuário.
AttributeStatement	Atributos do usuário.	Pode carregar e-mail, grupo, lotação, perfil.
Signature	Assinatura digital XML.	Sem validação correta, há risco de assertion forjada.

9.3. SAML x OIDC

Critério	SAML	OpenID Connect
Formato	XML e SAML Assertions.	JSON/JWT e endpoints REST-like.
Uso típico	SSO corporativo, aplicações enterprise, legadas, SaaS tradicionais.	Aplicações web/mobile modernas, APIs, ecossistema OAuth2.
Termos comuns	IdP e SP.	OpenID Provider e Relying Party/Client.
Objeto principal	Assertion SAML.	ID Token e UserInfo, além de access token OAuth2.
Transporte comum	Redirect/POST bindings no navegador.	Redirecionamentos OAuth2, token endpoint, UserInfo endpoint.

Critério	SAML	OpenID Connect
Pegadinha	Não é sinônimo de LDAP nem de certificado digital.	Não é apenas OAuth2; adiciona identidade/autenticação.

10. JWT, Bearer Token e armazenamento no cliente

10.1. JWT em três partes

JWT - JSON Web Token é um formato compacto para transmitir claims entre partes. Visualmente costuma ter três partes separadas por pontos: header.payload.signature. O header indica tipo e algoritmo; o payload contém claims; a signature permite verificar integridade e autenticidade.

JWT assinado pode ser lido por qualquer pessoa que tenha o token, salvo se também estiver cifrado. Portanto, não coloque segredo sensível no payload apenas porque o token é assinado.

10.2. Bearer token

Bearer token significa, literalmente, token do portador. Quem porta o token pode usá-lo, salvo se houver mecanismo adicional de prova de posse. Por isso, access tokens devem ser protegidos em armazenamento e transporte. Em HTTP, o uso típico é no cabeçalho Authorization: Bearer <token>.

Memorização: bearer = portador. É como um crachá sem biometria: se alguém rouba, consegue tentar usar. Proof-of-possession é quando o sistema exige provar posse de chave, não apenas apresentar o token.

10.3. Armazenamento em SPA Angular

Em aplicações SPA - Single Page Application, como Angular, tokens no navegador exigem cautela. LocalStorage é simples, mas expõe o token em caso de XSS. Cookies HttpOnly reduzem exposição a JavaScript, mas exigem proteção contra CSRF e configuração adequada de SameSite, Secure e escopo.

Para prova, o ponto é: não há solução mágica. Segurança depende do modelo de ameaça, do fluxo, da proteção contra XSS/CSRF, do tempo de vida do token, da rotação de refresh tokens, do uso de PKCE e da validação correta no backend.

11. Exemplo prático com Angular, Quarkus e API

Imagine um sistema interno de aprovação de pagamentos. O usuário acessa o frontend Angular. O Angular redireciona para o provedor de identidade corporativo usando Authorization Code Flow com PKCE. Após login e MFA, o cliente recebe tokens de forma adequada ao fluxo. O backend Quarkus expõe APIs protegidas. Cada chamada traz um access token.

A API valida a assinatura do token, o issuer, a audience, a expiração, os escopos e regras de negócio. Mesmo que o token seja válido, a API precisa verificar se o usuário pode acessar aquele recurso específico. Isso evita o erro clássico: autenticar bem e autorizar mal.

Camada	Responsabilidade	Erro perigoso
Angular	Iniciar fluxo, proteger rotas de interface, evitar exposição indevida de tokens.	Achar que esconder botão no front é autorização real.
IdP/Authorization Server	Autenticar, aplicar MFA, emitir tokens, gerenciar sessão.	Emitir tokens longos demais ou com escopos excessivos.
Quarkus/API	Validar token e aplicar autorização de negócio.	Confiar em claims sem validar assinatura/audience/issuer.
Banco de dados	Persistir dados e logs com controles de acesso.	Permitir consulta direta sem trilha ou segregação.

Camada	Responsabilidade	Erro perigoso
SIEM/logs	Monitorar eventos, falhas, anomalias, auditoria.	Não registrar eventos de autenticação e autorização relevantes.

12. Ataques e falhas comuns nesse tema

Falha/ataque	Descrição	Defesa principal
Credential stuffing	Tentativa com credenciais vazadas de outros serviços.	MFA, detecção de anomalia, bloqueio adaptativo, senhas únicas.
Brute force	Tentativas repetidas de senha.	Rate limiting, lockout inteligente, MFA, monitoramento.
Phishing	Engano para capturar credencial/token.	MFA resistente a phishing, FIDO2, educação, detecção.
MFA fatigue	Múltiplos pushes até o usuário aprovar por cansaço.	Number matching, limites, alertas, FIDO2.
Token theft	Roubo de access/refresh token.	TLS, armazenamento seguro, expiração curta, rotação, revogação.
Replay	Reuso de mensagem/token capturado.	Nonce, expiração, assinatura, TLS, proof-of-possession.
Open redirect	Redirect URI manipulado para capturar código/token.	Registro e validação estrita de redirect URI.
Broken access control	API não valida permissão sobre o recurso.	Autorização server-side, ownership, ABAC/RBAC, testes.
Session fixation	Atacante fixa sessão antes do login.	Regenerar identificador de sessão após autenticação.

13. Pegadinhas FGV de alto rendimento

- OAuth2 não é sinônimo de autenticação; é autorização. OIDC adiciona autenticação e identidade.
- ID Token não é para chamar API; access token é para API.
- Refresh token não é enviado ao resource server; ele é usado com o authorization server para obter novos access tokens.
- Bearer token precisa ser protegido: quem possui pode usar.
- SAML usa XML/assertions e é forte em SSO corporativo; OIDC usa JSON/JWT e é comum em aplicações modernas.
- MFA exige fatores diferentes; duas senhas não formam MFA real.
- SSO não significa mesma senha em todos os sistemas.
- Frontend não autoriza de verdade; autorização crítica deve ocorrer no backend/API.
- Validar apenas assinatura do JWT não basta: também valide issuer, audience, expiração, escopos e contexto.
- RBAC é papel; ABAC é atributo; DAC é discricionário pelo dono; MAC é mandatório por política central.

14. Checklist para memorizar antes das questões

1. Autenticação = identidade; autorização = permissão; auditoria/accounting = registro do que ocorreu.
2. IAM = processo corporativo de ciclo de vida de identidades e acessos.
3. MFA = fatores diferentes: saber, possuir, ser.
4. SSO = login único por confiança em IdP, não reutilização de senha.

5. OAuth2 = autorização delegada; OIDC = autenticação/identidade sobre OAuth2; SAML = federação/SSO com XML assertion.
6. Access token vai para API; ID token vai para client; refresh token volta ao authorization server.
7. PKCE protege Authorization Code Flow em clientes públicos.
8. JWT assinado é íntegro/autêntico, mas não necessariamente confidencial.
9. Scope limita ação; audience limita destinatário; issuer identifica emissor; exp limita validade; nonce/state mitigam repetição/troca de fluxo.
10. Em API, validar token é necessário, mas não suficiente: precisa autorizar o recurso e a ação.

15. Aprofundamento: fluxo OAuth2/OIDC sem pular etapas

15.1. Authorization Code + PKCE passo a passo

Este é o fluxo que mais merece atenção para aplicações modernas com usuário final. Ele aparece em frontends web, aplicações mobile e integrações com provedores corporativos. A FGV costuma cobrar não só a definição, mas a ordem lógica dos componentes: client, authorization endpoint, redirect URI, authorization code, token endpoint, access token e, em OIDC, ID Token.

Pense no fluxo como uma compra com retirada controlada: o usuário autoriza no balcão oficial; o cliente recebe um comprovante temporário; depois troca esse comprovante em um guichê protegido. PKCE adiciona uma prova de que quem está trocando o comprovante é a mesma aplicação que iniciou o pedido.

Passo	O que acontece	Termo que pode aparecer
1	O cliente gera code verifier e code challenge, além de state e, em OIDC, nonce.	PKCE, state, nonce.
2	O usuário é redirecionado ao authorization endpoint do Authorization Server.	Authorization endpoint.
3	O usuário autentica, eventualmente com MFA, e consente com escopos.	Authentication, consent, scope.
4	O Authorization Server redireciona de volta para a redirect URI registrada com authorization code e state.	Authorization code, redirect URI.
5	O client confere state; depois troca o authorization code no token endpoint, apresentando o code verifier.	Token endpoint, code verifier.
6	O Authorization Server valida o verifier contra o challenge e emite tokens.	Access token, refresh token, ID token se OIDC.
7	A API recebe access token e valida assinatura ou introspecção, issuer, audience, expiração e escopos.	Resource server, JWKS, introspection.

Ordem mental: redireciona para autorizar -> recebe código -> troca código por token. PKCE protege a troca do código. O token é usado na API. O ID Token, quando houver OIDC, informa identidade ao client.

15.2. Clientes públicos e confidenciais

No OAuth2, client não é necessariamente pessoa; é aplicação. A distinção crítica é se o client consegue manter segredo. Um backend em servidor controlado pode guardar client secret com relativa segurança. Um SPA Angular ou aplicativo mobile não consegue manter segredo de forma confiável, porque o código roda ou é distribuído no ambiente do usuário.

Tipo de cliente	Consegue guardar segredo?	Exemplo	Consequência de prova
Confidential client	Sim, em regra.	Backend Quarkus server-side chamando token endpoint.	Pode autenticar-se com client secret ou método mais forte.

Tipo de cliente	Consegue guardar segredo?	Exemplo	Consequência de prova
Public client	Não de forma confiável.	SPA Angular, app mobile, CLI distribuído.	Deve usar Authorization Code + PKCE; não confie em client secret embutido.

15.3. Redirect URI, state e open redirect

Redirect URI é o endereço para o qual o Authorization Server envia o usuário de volta depois da autorização. Como esse ponto carrega authorization code, erro de validação pode vazar o código para atacante. Por isso, redirect URIs devem ser previamente registradas e validadas de modo estrito.

State é um valor imprevisível gerado pelo client para correlacionar a resposta com a requisição original. Se o state que volta não corresponde ao state enviado, a aplicação deve recusar. Em prova, state é fortemente associado a mitigação de **CSRF** e troca de resposta no fluxo OAuth/OIDC.

Termo	Função	Erro típico
redirect_uri	Destino autorizado para retorno do authorization code.	Aceitar qualquer domínio ou correspondência parcial frouxa.
state	Correlaciona requisição e resposta; mitiga CSRF.	Não validar ou aceitar state ausente.
nonce	Correlaciona autenticação OIDC e reduz replay no ID Token.	Confundir com salt de senha.
code verifier	Segredo efêmero do PKCE apresentado na troca do código.	Armazenar de forma indevida ou não conferir challenge.

16. Aprofundamento: validação de tokens em APIs

16.1. Validação mínima que a API deve fazer

Uma API não deve simplesmente decodificar um JWT e confiar no payload. Decodificar é apenas transformar Base64URL em JSON. Validar é verificar se o token é autêntico, íntegro, atual, destinado àquela API e suficiente para a ação solicitada.

Em arquitetura com Quarkus, Spring, Node ou API Gateway, essa validação pode ser feita por middleware, biblioteca de segurança, gateway ou resource server. Mas a responsabilidade lógica permanece: se a API aceita token inválido ou permissivo demais, o risco é da aplicação.

Verificação	Pergunta	Por que importa
Assinatura	Foi emitido por chave confiável?	Impede token forjado ou alterado.
iss - issuer	O emissor é o provedor esperado?	Evita aceitar token de outro ambiente/tenant.
aud - audience	O token foi emitido para esta API?	Evita uso de token destinado a outro serviço.
exp/nbf/iat	Está dentro da janela temporal?	Evita token vencido, prematuro ou suspeito.
scope/roles	O token autoriza esta ação?	Evita consulta virar aprovação ou leitura virar escrita.
contexto de negócio	O usuário pode esse recurso específico?	Evita IDOR/acesso horizontal.
revogação/introspecção	O token ainda é aceitável?	Importante para tokens opacos, logout e incidentes.

16.2. JWKS e introspecção

JWKS - JSON Web Key Set é um conjunto de chaves públicas usado para validar assinaturas de tokens JWT. O resource server baixa as chaves do emissor confiável e verifica a assinatura localmente. Isso é comum quando tokens são JWT assinados.

Introspection é uma consulta ao Authorization Server para saber se um token está ativo e quais metadados possui. É comum com tokens opacos ou em ambientes que preferem validação central. A prova pode contrapor os dois modelos: JWT permite validação local, enquanto token opaco geralmente exige introspecção ou consulta ao emissor.

Modelo	Vantagem	Cuidado
JWT validado por JWKS	Menos chamadas ao authorization server; validação local eficiente.	Revogação imediata é mais difícil se o token ainda não expirou.
Token opaco + introspecção	Estado/revogação central mais simples.	Mais dependência de disponibilidade/latência do authorization server.
Sessão server-side	Revogação e controle central por sessão.	Escalabilidade e compartilhamento de sessão exigem desenho adequado.

16.3. Session, cookie e token

Sessão é o estado autenticado/autorizado mantido após login. Cookie é um mecanismo de armazenamento/envio automático pelo navegador. Token é um artefato que carrega ou referencia autorização/identidade. Eles podem coexistir: uma aplicação web pode usar cookie de sessão; uma SPA pode usar access token; um gateway pode transformar um token externo em sessão interna.

Objeto	É o quê?	Risco típico
Sessão	Estado de autenticação mantido por servidor ou client.	Sessão longa, sem expiração, sem revogação ou sem rotação.
Cookie	Mecanismo HTTP de armazenamento/envio automático.	CSRF se mal configurado; roubo se sem Secure/HttpOnly quando aplicável.
Access token	Credencial de autorização para API.	Roubo por XSS, vazamento em log, validade longa.
Refresh token	Credencial para obter novos access tokens.	Altíssimo impacto se roubado; exige rotação e proteção.

17. Termos adjacentes que podem aparecer na FGV

17.1. LDAP, Active Directory, Kerberos e Keycloak

Embora o foco da aula seja IAM, MFA, SSO, OAuth2, OIDC e SAML, provas de TI costumam misturar ferramentas e protocolos próximos. Você não precisa estudar administração profunda de cada um aqui, mas deve reconhecer o papel de cada termo.

Termo	Expansão/ideia	Associação de prova
LDAP	Lightweight Directory Access Protocol	Protocolo para consulta e manutenção de diretórios; muito usado para usuários/grupos.
Active Directory	Serviço de diretório da Microsoft	Gerencia identidades, grupos, políticas e autenticação em domínio Windows.
Kerberos	Protocolo de autenticação por tickets	Usa KDC e tickets; evita envio repetido de senha pela rede.
Keycloak	Solução de IAM/IdP open source	Implementa SSO e suporta OIDC/OAuth2 e SAML.
PAM	Privileged Access Management	Gestão de contas privilegiadas, cofre de senhas, sessão monitorada e just-in-time access.
IGA	Identity Governance and Administration	Governança de identidades: recertificação, SoD, workflow de acesso.

17.2. Segregação de funções - SoD

SoD - **Segregation of Duties** é um princípio de controle interno: **uma mesma pessoa não deve acumular funções incompatíveis em processo crítico**. Em sistemas financeiros e judiciais, isso aparece em aprovar e executar, cadastrar e liberar, desenvolver e homologar em produção, ou solicitar e aprovar o próprio acesso.

No IAM, SoD pode ser preventivo, impedindo a concessão de perfis conflitantes, ou detectivo, apontando conflito em revisões periódicas. A FGV pode cobrar isso como governança de acessos, risco operacional ou conformidade.

Associação com sua experiência no banco: esteiras, aprovação, documentação e controles internos não são burocracia solta; em prova, eles viram segregação de funções, trilha de auditoria, menor privilégio, necessidade de aprovação e revisão de acesso.

17.3. Inferência por inglês técnico

Palavra	Tradução útil	Inferência
Bearer	Portador	Quem porta o token consegue usá-lo.
Claim	Afirmção/reivindicação	Dado afirmado no token/assertion.
Assertion	Afirmção	SAML Assertion é a afirmação de identidade/atributo emitida pelo IdP.
Audience	Audiência/destinatário	Quem deve aceitar o token.
Issuer	Emissor	Quem emitiu token/assertion.
Subject	Sujeito	A identidade sobre a qual o token fala.
Scope	Escopo	Limite de atuação do cliente.
Proof	Prova	PKCE e proof-of-possession exigem prova adicional, não só apresentação.
Refresh	Renovar	Refresh token renova access token.
Revocation	Revogação	Invalidar token, sessão ou credencial.

17.4. Cenários de prova comentados antes da bateria

Use os cenários abaixo como treino de leitura. A FGV costuma esconder a resposta não no termo técnico isolado, mas na relação entre atores, finalidade e objeto de segurança.

Cenário	Raciocínio correto	Armadilha provável
Usuário autenticou com MFA, mas acessa processo de outra unidade.	Problema de autorização/controle de acesso por recurso.	Responder autenticação, MFA ou criptografia.
Aplicação quer saber quem é o usuário e obter e-mail padronizado.	OIDC com ID Token e/ou UserInfo.	Responder OAuth2 puro.
Serviço backend chama API sem usuário final.	Client Credentials.	Responder Authorization Code.
Aplicação enterprise antiga integra SSO via XML assinado.	SAML Assertion entre IdP e SP.	Responder JWT ou LDAP.
Token assinado emitido para API A chega à API B.	Recusar por audience incompatível.	Aceitar por assinatura válida.
Token roubado no navegador por XSS.	Bearer token exposto pode ser usado pelo portador.	Achar que assinatura impede uso por atacante.
Dois campos de senha em sequência.	Não é MFA real: mesmo fator de conhecimento.	Confundir duas etapas com dois fatores.
Usuário aprova push de MFA que não iniciou.	MFA fatigue/push bombing.	Classificar como brute force comum.

17.5. Tabela-mãe para revisão de véspera

Tema	Palavra-chave	Resposta curta de prova
IAM	Lifecycle	Entrar, mudar, sair, revisar e revogar acessos.
Autenticação	Identity proof	Provar que controla credencial/autenticador.
Autorização	Permission	Decidir ação permitida sobre recurso.
MFA	Different factors	Fatores independentes; duas senhas não bastam.
SSO	One login	Login único por confiança em IdP.
OAuth2	Delegated authorization	Cliente acessa recurso com token limitado.
OIDC	Identity layer	OAuth2 + autenticação + ID Token.
SAML	XML assertion	SSO/federação com assertion XML assinada.
JWT	Claims	Formato compacto; assinado não implica cifrado.
PKCE	Code protection	Protege troca do authorization code.
Bearer	Holder	Quem porta usa; proteja contra vazamento.
RBAC	Roles	Permissões por papéis.
ABAC	Attributes	Permissões por atributos/contexto.

17.6. Caderno de erros previsto

Quando errar questão desta aula, classifique o erro em uma destas gavetas. Isso reduz repetição de erro e acelera revisão ativa.

Gaveta de erro	Como aparece	Correção rápida
Conceito	Confundi OAuth2 com OIDC ou SAML com JWT.	Reescrever a diferença em uma frase.
Termo parecido	Troquei nonce, salt, state, scope ou audience.	Usar a tabela de associação rápida.
Leitura apressada	O enunciado falava de API, mas marquei login.	Perguntar: o problema é quem é, o que pode ou como confia?
Pegadinha FGV	Assinatura válida parecia suficiente.	Lembrar: validar token exige conjunto de condições.
Vivência prática mal traduzida	No trabalho a ferramenta faz automático, mas a prova pergunta o conceito.	Separar ferramenta de protocolo e protocolo de objetivo.

18. Questões autorais no padrão FGV - sem gabarito

As questões a seguir são autorais e foram construídas para simular o estilo FGV: enunciados mais situacionais, alternativas plausíveis, mistura de conceito com aplicação e distratores que exploram confusão entre termos. Não há gabarito nesta seção.

1. Em um tribunal, um novo portal de serviços internos passa a utilizar um provedor corporativo de identidade. Após autenticar-se uma vez, o servidor consegue acessar sistemas de gestão documental, chamados administrativos e painel de indicadores sem redigitar sua senha. O desenho também permite revogar o acesso de um servidor desligado em ponto central. Nesse cenário, a descrição mais adequada é a de que o portal adotou:

- (A) Single Sign-On baseado em provedor de identidade, pois a autenticação é centralizada e reutilizada por aplicações confiáveis.
- (B) reutilização segura de senha, pois o mesmo segredo criptográfico é copiado entre aplicações distintas.
- (C) controle de acesso discricionário, pois cada usuário passa a decidir quais sistemas internos aceitarão sua identidade.
- (D) hash federado de credenciais, pois cada sistema valida localmente o resumo criptográfico da senha corporativa.
- (E) criptografia ponta a ponta de sessão, pois a autenticação única elimina a necessidade de autorização nas aplicações.

2. Uma equipe desenvolve uma API de processos judiciais. O usuário está autenticado no provedor corporativo, mas a API permite que ele altere o parâmetro idProcesso na URL e consulte processo de outra unidade para a qual não possui permissão. O problema descrito decorre, principalmente, de falha de:

- (A) autenticação multifator, pois o segundo fator deveria impedir a troca de identificadores na URL.
- (B) confidencialidade do token, pois o JWT deveria ser cifrado para impedir a leitura do identificador do processo.
- (C) disponibilidade da sessão, pois a API deveria manter a sessão ativa por menos tempo.
- (D) autorização no lado servidor, pois autenticar o usuário não basta para permitir acesso a qualquer recurso.
- (E) não repúdio, pois a API deveria exigir assinatura digital do usuário em cada consulta.

3. Uma aplicação Angular utiliza OAuth2 para acessar uma API REST. O gestor do projeto afirma que OAuth2 é, por definição, um protocolo de autenticação de usuário final e que, portanto, dispensa qualquer camada adicional de identidade. À luz dos padrões modernos de identidade, a afirmação é:

- (A) correta, pois OAuth2 substitui integralmente SAML, OpenID Connect e mecanismos de MFA.
- (B) incorreta, pois OAuth2 é primariamente um framework de autorização delegada; para autenticação federada padronizada, usa-se OpenID Connect sobre OAuth2.
- (C) correta, desde que o access token esteja no formato JWT, pois todo JWT é obrigatoriamente um ID Token.
- (D) incorreta, pois OAuth2 só é aplicável a autenticação máquina-máquina e não permite autorização de usuários finais.
- (E) correta, se o cliente usar refresh token, pois a renovação do token prova a identidade civil do usuário.

4. No fluxo Authorization Code com PKCE, uma aplicação pública gera um code verifier e envia ao servidor de autorização um code challenge derivado dele. Posteriormente, ao trocar o código de autorização por tokens, apresenta o verifier. O objetivo principal desse mecanismo é:

- (A) permitir que a API de recurso decifre o conteúdo do access token sem consultar o emissor.

- (B) substituir completamente o uso de TLS nas chamadas ao authorization endpoint e ao token endpoint.
- (C) garantir que o usuário final nunca precise autenticar-se perante o provedor de identidade.
- (D) transformar um access token opaco em JWT assinado por chave assimétrica.
- (E) reduzir o risco de troca indevida de um authorization code interceptado por quem não possui o verifier correspondente.

5. Considere um JWT recebido por uma API interna. O token possui assinatura válida, mas foi emitido para outra audiência, correspondente a uma API distinta. Segundo boas práticas de validação de tokens, a API interna deve:

- (A) aceitar o token, pois assinatura válida é suficiente para comprovar qualquer autorização corporativa.
- (B) aceitar o token apenas se o campo sub identificar um usuário ativo no diretório corporativo.
- (C) recusar o token, pois a claim aud deve corresponder ao destinatário esperado do token.
- (D) converter o token em refresh token, pois access tokens não possuem restrição de audiência.
- (E) ignorar a audiência e validar apenas se o algoritmo informado no header é simétrico.

6. Um analista propõe substituir senhas por dois campos: uma senha principal e uma senha secundária, ambas memorizadas pelo usuário. Ele afirma que, por haver duas etapas, a solução é autenticação multifator. A avaliação tecnicamente adequada é que a proposta:

- (A) não caracteriza MFA robusto, pois usa dois elementos do mesmo fator de conhecimento.
- (B) caracteriza MFA, pois qualquer autenticação em duas telas distintas envolve fatores independentes.
- (C) é equivalente a FIDO2, pois duas senhas criam prova criptográfica de posse de hardware.
- (D) elimina ataques de phishing, pois o invasor não consegue solicitar duas informações ao usuário.
- (E) substitui a necessidade de gestão de identidade, pois as senhas passam a fazer autorização e auditoria.

7. Um órgão utiliza SAML para integrar um sistema de gestão de pessoas a um serviço externo. No fluxo, o provedor de identidade autentica o usuário e envia ao prestador do serviço uma afirmação XML assinada com dados de autenticação e atributos. Nesse contexto, o objeto enviado pelo provedor de identidade é denominado:

- (A) access scope.
- (B) refresh token.
- (C) authorization code.
- (D) SAML assertion.
- (E) PKCE verifier.

8. Uma aplicação recebe um ID Token em um fluxo OpenID Connect e pretende utilizá-lo diretamente no cabeçalho Authorization para acessar uma API de pagamentos. A prática é inadequada porque:

- (A) o ID Token nunca pode estar em formato JWT, ao contrário do access token.
- (B) o ID Token é destinado ao client/Relying Party como prova de autenticação, enquanto a API deve receber e validar access token apropriado.
- (C) o ID Token é sempre opaco e só pode ser lido pelo navegador do usuário final.
- (D) o access token serve apenas para logout e não para acesso a recursos protegidos.
- (E) o uso de ID Token em APIs é obrigatório quando o usuário autenticou com MFA.

9. Em um sistema corporativo, as permissões de acesso são concedidas com base no cargo do usuário, no órgão de lotação, no horário da requisição, na classificação do documento e no nível de risco calculado dinamicamente. O modelo de controle de acesso que melhor representa essa decisão é:

- (A) DAC, pois o dono do documento decide livremente o acesso em tempo de execução.
- (B) MAC, pois todo acesso baseado em cargo é necessariamente mandatário.
- (C) RBAC puro, pois basta mapear cada cargo para uma lista fixa de permissões.
- (D) SSO, pois a decisão ocorre depois de uma autenticação centralizada.
- (E) ABAC, pois atributos do sujeito, recurso, ação e ambiente influenciam a decisão.

10. Um token do tipo bearer é armazenado em local acessível por scripts no navegador. Uma vulnerabilidade XSS permite que um atacante leia esse token e o envie para fora da aplicação. A principal razão pela qual esse cenário é crítico é que:

- (A) tokens bearer conferem acesso ao portador, de modo que sua exposição pode permitir uso indevido enquanto forem válidos.
- (B) tokens bearer são sempre refresh tokens de validade permanente.
- (C) um token bearer contém obrigatoriamente a senha do usuário em texto claro.
- (D) todo bearer token é cifrado com chave pública do atacante por padrão.
- (E) a presença de XSS impede qualquer validação de assinatura pelo resource server.

11. Uma organização deseja que aplicações internas não armazenem senhas locais dos usuários, que a política de MFA seja centralizada e que a revogação de acesso ocorra no ponto corporativo de identidade. A decisão arquitetural mais alinhada a esse objetivo é:

- (A) adotar federação/SSO com provedor de identidade confiável e integração das aplicações por SAML ou OIDC, conforme o caso.
- (B) replicar periodicamente a base de senhas para todas as aplicações, reduzindo a dependência do provedor central.
- (C) permitir que cada aplicação defina seu próprio formato de senha e seus próprios critérios de MFA.
- (D) substituir autenticação por autorização em banco de dados, pois permissões SQL resolvem o login do usuário final.
- (E) usar apenas criptografia simétrica entre navegador e banco de dados, eliminando IdP e sessões.

12. No contexto de OAuth2, um backend Quarkus precisa chamar uma API de cadastro em nome da própria aplicação, sem usuário final interativo. O fluxo mais apropriado, em regra, é:

- (A) Implicit Flow, por retornar o token diretamente no navegador.
- (B) Resource Owner Password Credentials, por exigir que o backend colete a senha do usuário final.
- (C) Authorization Code sem PKCE, por depender sempre de interação humana.
- (D) Client Credentials, por representar autenticação/autorização de cliente para comunicação máquina-máquina.
- (E) SAML POST Binding, por ser o único fluxo possível para APIs REST modernas.

13. Um JWT contém claims como iss, sub, aud e exp. Em uma revisão de segurança, um desenvolvedor afirma que basta decodificar o payload Base64URL e confiar nos dados, pois JWT é um formato padronizado. A orientação correta é:

- (A) confiar nos dados se o campo exp existir, ainda que a assinatura não seja validada.

- (B) validar assinatura, emissor, audiência, expiração e demais condições antes de confiar nas claims.
 - (C) remover a assinatura para evitar dependência de chave pública no resource server.
 - (D) aceitar qualquer algoritmo informado no header, inclusive se for incompatível com a política do emissor.
 - (E) usar o campo sub como escopo de autorização, pois subject e scope são semanticamente idênticos.
14. Um usuário recebe diversas notificações push de MFA sem ter iniciado login. Cansado, aprova uma delas. O invasor consegue acessar a conta porque já possuía a senha. O ataque descrito é mais bem classificado como:
- (A) ataque de downgrade criptográfico contra o algoritmo de assinatura do token.
 - (B) controle de acesso mandatório aplicado indevidamente ao usuário final.
 - (C) MFA fatigue ou bombardeio de notificações de autenticação.
 - (D) prova de posse baseada em chave assimétrica resistente a phishing.
 - (E) ataque de colisão contra função hash de senha.
15. Em uma integração SAML, o Service Provider recebe uma assertion assinada. Para reduzir o risco de aceitar uma assertion em contexto indevido, o SP deve, entre outros controles:
- (A) aceitar qualquer assertion assinada por qualquer autoridade certificadora pública.
 - (B) ignorar janela temporal, pois a assinatura digital já impede replay por tempo indeterminado.
 - (C) substituir a validação de assinatura por verificação visual do XML recebido.
 - (D) validar assinatura, emissor confiável, audiência, destino, condições temporais e relação com a solicitação quando aplicável.
 - (E) converter a assertion XML em refresh token OAuth2 sem validação adicional.
16. Uma equipe define que usuários com papel 'analista' podem consultar documentos, usuários com papel 'gestor' podem aprovar solicitações, e usuários com papel 'administrador' podem parametrizar o sistema. O modelo predominante nessa descrição é:
- (A) RBAC - Role-Based Access Control.
 - (B) ABAC - Attribute-Based Access Control.
 - (C) DAC - Discretionary Access Control.
 - (D) MAC - Mandatory Access Control.
 - (E) PKCE - Proof Key for Code Exchange.
17. Uma aplicação web usa OIDC. Durante a resposta de autenticação, o client verifica um valor nonce enviado originalmente na requisição e recebido no ID Token. A finalidade dessa verificação é principalmente:
- (A) garantir que o access token tenha escopos administrativos.
 - (B) substituir a validação da assinatura do ID Token.
 - (C) permitir que o usuário altere seu subject sem novo login.
 - (D) reduzir risco de replay e associar a resposta à solicitação de autenticação iniciada pelo client.
 - (E) transformar autenticação federada em controle de acesso discricionário.
18. Um comitê de segurança avalia duas alternativas para SSO: SAML e OIDC. A comparação correta é:

- (A) SAML usa assertions XML e é comum em SSO corporativo; OIDC usa OAuth2 como base e ID Token, frequentemente em formato JWT.
- (B) SAML e OIDC são algoritmos de hash usados para armazenar senha com salt.
- (C) OIDC é exclusivo para redes locais sem internet, enquanto SAML só funciona em APIs REST JSON.
- (D) SAML substitui a necessidade de assinatura digital, enquanto OIDC não permite claims de identidade.
- (E) OIDC é uma versão antiga do OAuth1 e SAML é um perfil obrigatório do TLS.

19. Em uma API, o desenvolvedor valida a assinatura do access token, mas não verifica se o escopo inclui 'pagamento:aprovar'. Com isso, tokens emitidos apenas para consulta passam a aprovar pagamentos. O erro está relacionado à ausência de validação de:

- (A) issuer, pois todo erro de autorização decorre de emissor inválido.
- (B) audience, pois escopo e audiência são sempre a mesma claim.
- (C) nonce, pois todo access token precisa de nonce para autorização de negócio.
- (D) algoritmo de hash da senha do usuário.
- (E) escopo/permissão necessária para a ação solicitada.

20. Uma instituição usa autenticação centralizada, revisão periódica de acessos, desativação automática de contas inativas, segregação de funções e trilhas de auditoria para acessos privilegiados. O conjunto dessas práticas se enquadra melhor em:

- (A) criptografia simétrica de banco de dados.
- (B) IAM e governança de acessos.
- (C) compressão de tokens JWT.
- (D) balanceamento de carga em APIs REST.
- (E) normalização de dados cadastrais.

19. Gabarito resumido

1-A | 2-D | 3-B | 4-E | 5-C | 6-A | 7-D | 8-B | 9-E | 10-A | 11-A | 12-D | 13-B | 14-C | 15-D | 16-A | 17-D | 18-A | 19-E | 20-B

20. Comentários completos

Questão 1 - Gabarito: A

A alternativa A está correta porque descreve SSO real: autenticação centralizada em provedor de identidade e reutilização de sessão/tokens/assertions por aplicações confiáveis. B está errada porque copiar ou reutilizar senha entre aplicações não é SSO e aumenta risco. C confunde com DAC: o usuário não decide quais sistemas confiarão em sua identidade. D inventa 'hash federado' e ignora que sistemas não devem validar senha localmente. E confunde autenticação única com criptografia e ainda sugere que autorização seria desnecessária. Erro provável: confusão entre SSO e senha única.

Questão 2 - Gabarito: D

D está correta: o usuário pode estar autenticado e, ainda assim, não estar autorizado a acessar aquele idProcesso. A falha é broken access control/ausência de autorização server-side por recurso. A está errada porque MFA não valida ownership do recurso. B pode ser uma preocupação de confidencialidade, mas não resolve acesso indevido horizontal. C trata disponibilidade, fora do núcleo. E assinatura digital por consulta não é a solução primária. Erro provável: confusão entre autenticação e autorização.

Questão 3 - Gabarito: B

B está correta: OAuth2 é framework de autorização delegada; OIDC acrescenta camada de identidade e autenticação. A está errada porque OAuth2 não substitui tudo nem é autenticação pura. C erra ao dizer que todo JWT é ID Token; JWT é formato. D é falsa porque OAuth2 também trata autorização com usuário final. E confunde refresh token com prova de identidade civil. Erro provável: memorização incompleta de OAuth2.

Questão 4 - Gabarito: E

E está correta: PKCE reduz o risco de um authorization code interceptado ser trocado por tokens, pois o atacante não tem o code verifier. A erra sobre decifrar access token. B é gravemente errada: TLS continua necessário. C erra porque o usuário ainda pode autenticar-se. D confunde PKCE com formato/assinatura do token. Erro provável: lacuna de teoria sobre PKCE.

Questão 5 - Gabarito: C

C está correta: audience - aud - indica o destinatário pretendido do token. API com audiência diferente deve recusar, ainda que a assinatura seja válida. A ignora validações além da assinatura. B valida existência de usuário, mas isso não substitui aud. D inventa conversão de token. E ignora a validação de destinatário. Erro provável: achar que assinatura válida basta.

Questão 6 - Gabarito: A

A está correta: duas senhas são dois passos, mas não dois fatores independentes. Ambas pertencem ao fator conhecimento. B confunde etapas com fatores. C confunde senha com FIDO2, que usa criptografia/chave. D é falsa: phishing pode coletar múltiplas informações. E mistura autenticação com IAM, autorização e auditoria. Erro provável: confusão entre 2FA/MFA e autenticação em duas telas.

Questão 7 - Gabarito: D

D está correta: no SAML, o IdP emite uma SAML assertion, geralmente XML assinada, com declarações sobre autenticação e atributos. A é termo de escopo OAuth. B é token de renovação OAuth. C é código temporário OAuth. E é segredo do fluxo PKCE. Erro provável: confusão entre token OAuth e assertion SAML.

Questão 8 - Gabarito: B

B está correta: ID Token é consumido pelo client/RP para identidade/authenticação; APIs devem receber access tokens apropriados. A é falsa: ID Token frequentemente é JWT. C é falsa: ID Token não é necessariamente opaco.

D inverte: access token serve para acessar recursos protegidos. E MFA não torna ID Token substituto de access token. Erro provável: confusão entre ID Token e Access Token.

Questão 9 - Gabarito: E

E está correta: a decisão usa atributos de usuário, recurso, ação e ambiente, caracterizando ABAC. A seria dono do recurso decidindo. B não basta ser cargo para ser MAC; MAC envolve política central obrigatória e rótulos/níveis. C é RBAC puro, insuficiente para horário, risco e classificação. D é mecanismo de login único, não modelo de autorização. Erro provável: identificar qualquer cargo como RBAC e ignorar atributos.

Questão 10 - Gabarito: A

A está correta: bearer token dá acesso a quem o porta. Se XSS rouba token, o invasor pode usá-lo enquanto válido. B erra ao dizer que bearer sempre é refresh token permanente. C é falsa: token não contém obrigatoriamente senha. D inventa cifragem para o atacante. E a API ainda pode validar assinatura, mas isso não impede uso por quem roubou token válido. Erro provável: subestimar o sentido de 'bearer'.

Questão 11 - Gabarito: A

A está correta: federação/SSO com IdP central permite centralizar autenticação, MFA, revogação e confiança entre sistemas, por SAML ou OIDC. B aumenta risco por replicação de senhas. C fragmenta política. D confunde autorização SQL com autenticação federada. E criptografia de transporte/armazenamento não substitui IAM. Erro provável: tratar login como problema isolado de cada aplicação.

Questão 12 - Gabarito: D

D está correta: Client Credentials é o fluxo típico para comunicação máquina-máquina sem usuário final. A é fluxo antigo/browser. B exige senha do usuário e é desencorajado. C depende de interação de usuário. E SAML pode fazer SSO, mas não é o fluxo típico de OAuth para API REST máquina-máquina. Erro provável: não associar client credentials a serviço-serviço.

Questão 13 - Gabarito: B

B está correta: JWT deve ser validado quanto à assinatura, issuer, audience, expiração e condições relevantes antes de confiar nas claims. A é perigosa: exp sem assinatura não vale. C remover assinatura destrói integridade. D aceitar qualquer alg abre risco. E confunde subject com scope. Erro provável: achar que decodificar JWT é validar JWT.

Questão 14 - Gabarito: C

C está correta: o ataque descrito é MFA fatigue, push bombing ou bombardeio de notificações. A fala de downgrade criptográfico, fora do caso. B trata MAC, sem relação. D descreve defesa forte, não o ataque. E fala colisão de hash, sem relação. Erro provável: não conhecer ataques contra MFA push.

Questão 15 - Gabarito: D

D está correta: SP deve validar assinatura, emissor, audiência, destino, condições temporais e relação com a requisição, quando houver. A é errada: não basta qualquer AC pública. B ignora replay e validade temporal. C é absurda: validação visual não substitui assinatura. E mistura SAML com OAuth sem validação. Erro provável: achar que assinatura isolada basta.

Questão 16 - Gabarito: A

A está correta: permissões por papel - analista, gestor, administrador - caracterizam RBAC. B exigiria atributos dinâmicos/contextuais. C seria dono do recurso decidindo. D envolveria política central obrigatória/rótulos. E é mecanismo de OAuth, não modelo de acesso. Erro provável: confundir modelo de acesso com protocolo.

Questão 17 - Gabarito: D

D está correta: nonce associa a resposta ao pedido de autenticação e reduz risco de replay. A confunde nonce com escopo. B é falsa: nonce não substitui assinatura. C é absurda: subject não deve ser alterado livremente pelo usuário. E mistura OIDC com DAC. Erro provável: confundir nonce com salt ou scope.

Questão 18 - Gabarito: A

A está correta: SAML usa XML/assertions e é comum em SSO corporativo; OIDC é camada de identidade sobre OAuth2 e usa ID Token, geralmente JWT. B confunde protocolos com hash de senha. C inverte e inventa restrições. D é falso: SAML usa assinatura digital e OIDC permite claims. E mistura OAuth1, TLS e OIDC sem sentido. Erro provável: memorização superficial das siglas.

Questão 19 - Gabarito: E

E está correta: a API não verificou se o token possui escopo/permissão para aprovar pagamento. A issuer é importante, mas não é o erro descrito. B audience e scope não são a mesma coisa. C nonce é típico de OIDC, não autorização de ação em access token. D hash da senha não se relaciona ao token recebido. Erro provável: validar token sem validar permissão de negócio.

Questão 20 - Gabarito: B

B está correta: ciclo de vida, revisão de acessos, desativação de contas, segregação de funções e trilha de auditoria pertencem a IAM e governança de acessos. A é criptografia, não gestão de identidade. C é formato/otimização inexistente. D é infraestrutura. E é dados, não segurança de acesso. Erro provável: reduzir IAM a login.

21. Flashcards finais

Fronte	Verso
OAuth2 é autenticação?	Não. É autorização delegada. Para autenticação padronizada sobre OAuth2, use OIDC.
OIDC adiciona o quê ao OAuth2?	Camada de identidade/authenticação, escopo openid e ID Token.
SAML usa qual formato?	XML, com assertions assinadas entre IdP e SP.
Access token serve para quê?	Acessar recurso/API conforme escopos e condições.
ID Token serve para quê?	Informar ao client/RP que o usuário autenticou e transportar claims de identidade.
Refresh token vai para a API?	Não. Deve ser usado com o authorization server para obter novo access token.
MFA é duas senhas?	Não. MFA exige fatores diferentes.
SSO é mesma senha?	Não. SSO é confiança em autenticação central/federada.
Bearer significa o quê?	Portador. Quem porta o token pode usá-lo se não houver prova adicional.
JWT assinado é secreto?	Não necessariamente. Assinatura garante integridade/authenticidade; confidencialidade exige cifragem.
aud em JWT significa?	Audience: destinatário esperado do token.
iss em JWT significa?	Issuer: emissor do token.
sub em JWT significa?	Subject: sujeito/identidade a que o token se refere.
exp em JWT significa?	Expiration Time: tempo de expiração.
nonce serve para quê?	Evitar replay/correlacionar resposta de autenticação.
state em OAuth/OIDC serve para quê?	Correlacionar fluxo e mitigar CSRF/troca de resposta.
salt serve para quê?	Fortalecer armazenamento de senha antes do hash, tornando hashes únicos e resistentes a rainbow tables.
RBAC é baseado em quê?	Papéis/cargos.
ABAC é baseado em quê?	Atributos do sujeito, recurso, ação e ambiente.
MAC x DAC?	MAC = política central obrigatória; DAC = dono decide.

22. Mini revisão em 15 minutos

1. Leia a tabela de siglas e fale em voz alta: OAuth2 autorização, OIDC identidade, SAML assertion XML.
2. Revise a tabela ID Token x Access Token; esse é um dos pontos de maior ROI.
3. Releia as pegadinhas sobre MFA, SSO e JWT assinado não cifrado.
4. Resolva as 20 questões sem olhar o gabarito e marque as que você errou por conceito ou leitura apressada.
5. Crie no caderno de erros uma linha para cada confusão: nonce x salt x state; access token x ID token; RBAC x ABAC; OAuth2 x OIDC.

23. Fontes técnicas consultadas

As fontes abaixo foram utilizadas para calibrar conceitos técnicos e tendência de cobrança. A lista é objetiva, sem depender de material enciclopédico fora do edital.

- FGV Conhecimento - cadernos oficiais de provas de TI e Segurança da Informação, especialmente TJRR e TJRJ, para calibrar estilo de enunciado, alternativas e temas de autenticação/autorização.
- IETF RFC 6749 - The OAuth 2.0 Authorization Framework.

- IETF RFC 6750 - OAuth 2.0 Bearer Token Usage.
- IETF RFC 7636 - Proof Key for Code Exchange by OAuth Public Clients.
- IETF RFC 9700 - Best Current Practice for OAuth 2.0 Security.
- OpenID Foundation - OpenID Connect Core 1.0.
- OASIS - SAML 2.0 Technical Overview e especificações SAML V2.0.
- NIST SP 800-63B-4 - Digital Identity Guidelines: Authentication and Authenticator Management.
- IETF RFC 7519 - JSON Web Token (JWT).